

LAB TEST-2

AI ASSIST CODING

Name:P.Susmija

Roll no:2503A51L11

Batch no:24BTCAICSB19

Sub Group-i

TASK-1:

Top-3 words by frequency; tie-break lexicographically.

Prompt Given:

write a python program using AI for Counter and sort keys,Constraints & Notes: Correct tie-breaking.

Code Generated:

```
Users > Susmija > OneDrive > Desktop > AI > 1s.py > ...
from collections import Counter

def top_three_words(text: str):
    # Step 1: Normalize (lowercase + split by spaces)
    words = text.lower().split()

    # Step 2: Count word frequencies
    counter = Counter(words)

    # Step 3: Sort by frequency (descending) and lex order (ascending)
    sorted_words = sorted(counter.items(), key=lambda x: (-x[1], x[0]))

    # Step 4: Take top 3
    return sorted_words[:3]

# ----- Example Run -----
if __name__ == "__main__":
    sample_input = "to be or not to be that is the question"
    result = top_three_words(sample_input)
    print(result) # [('to', 2), ('be', 2), ('is', 1)]
```

Output:

```

PS C:\Users\Susmija> & C:/Users/Susmija/AppData/Local/Programs/Python/Python313/python.exe /Desktop/AI/1s.py
[('be', 2), ('to', 2), ('is', 1)]
PS C:\Users\Susmija> & C:/Users/Susmija/AppData/Local/Programs/Python/Python313/python.exe /Desktop/AI/1s.py
[('be', 2), ('to', 2), ('is', 1)]

```

TASK-2:

Implement capacity=2 LRU with get/put.

Data & Edge Cases:

Operations sequence provided

Prompted Given: write a python program to implement ordered dict from the collections

Code Generated:

```

sers > Susmija > OneDrive > Desktop > AI > 2s.py > ...
from collections import OrderedDict

class LRUCache:
    def __init__(self, capacity: int):
        self.cache = OrderedDict()
        self.capacity = capacity

    def get(self, key: int) -> int:
        if key not in self.cache:
            return -1
        # Move to end to mark as recently used
        self.cache.move_to_end(key)
        return self.cache[key]

    def put(self, key: int, value: int) -> None:
        if key in self.cache:
            # Update value and mark as recently used
            self.cache.move_to_end(key)
            self.cache[key] = value
        if len(self.cache) > self.capacity:
            # Pop the least recently used item (first one)
            self.cache.popitem(last=False)

# ----- Example Run -----
if __name__ == "__main__":
    ops = [("put",1,1),("put",2,2),("get",1),("put",3,3),("get",2),("get",3)]
    lru = LRUCache(2)

    results = []

```

```

def put(self, key: int, value: int) -> None:
    self.cache.move_to_end(key)
    self.cache[key] = value
    if len(self.cache) > self.capacity:
        # Pop the least recently used item (first one)
        self.cache.popitem(last=False)

----- Example Run -----
if __name__ == "__main__":
    ops = [("put",1,1),("put",2,2),("get",1),("put",3,3),("get",2),("get",3)]
    lru = LRUCache(2)

    results = []
    for op in ops:
        if op[0] == "put":
            lru.put(op[1], op[2])
            results.append(None) # Put returns nothing
        elif op[0] == "get":
            results.append(lru.get(op[1]))

    print(results)
    # Output: [None, None, 1, None, -1, 3]

```

Output:

```

PROBLEMS 55 OUTPUT DEBUG CONSOLE TERMINAL PORTS
PS C:\Users\Susmija> & C:/Users/Susmija/AppData/Local/Programs/Python/Python313/python.exe c:/Users/Susmija/OneDrive
/Desktop/AI/2s.py
[None, None, 1, None, -1, 3]
PS C:\Users\Susmija>

```